Clustering Fundamentals and Customer Segmentation

Clustering basics

Clustering is an **unsupervised machine learning technique** used to divide data into groups such that:

Objects within the same group are similar, while objects belonging to different groups are dissimilar.

For example, suppose a shopping mall has 1,000 customers.

We don't know beforehand which customer belongs to which category.

We might want to discover groups such as:

- **High-income, high-spending customers**
- **High-income, low-spending customers**
- **Low-income, high-spending customers**
- **Low-income, low-spending customers**

This is a clustering problem.

Real-World Applications of Clustering

Marketing

Customer segmentation:

E-commerce

Group customers based on:

- purchase frequency
- spending
- product preferences

Healthcare

Group patients with similar characteristics.

Image Processing

Group pixels with similar colours.

NLP

Group documents with similar topics.

Banking

Group customers according to financial behaviour.

Distance Metrics

Usually, we calculate a distance to decide which points are similar which are different

Smaller distance → more similar.

Larger distance → less similar.

Euclidean Distance

The most commonly used distance with K-Means.

For two points:

$$A = (x_1, y_1)$$

and

$$B = (x_2, y_2)$$

Euclidean distance is:

$$d(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Manhattan Distance

Another important distance metric.

Instead of measuring the straight-line distance, Manhattan distance measures the total movement along each dimension.

$$d(A, B) = |x_2 - x_1| + |y_2 - y_1|$$

K-Means Clustering

K-Means is one of the most popular clustering algorithms.
Basic idea :

K-means partitions N observations into K groups.
For each cluster/group (Cj), it computes a centroid μj

Mathematically, the centroid is the mean of the points.

For one dimension:

$$\mu = \frac{x_1 + x_2 + \cdots + x_n}{n}$$

For two dimensions:

$$\mu_x = \frac{x_1 + x_2 + \cdots + x_n}{n}$$

$$\mu_y = \frac{y_1 + y_2 + \cdots + y_n}{n}$$

K-Means Algorithm

1. Choose the value of K.
2. Choose K initial centroids.
3. Assign each point to the closest centroid by using distance metrices.
4. Recalculate each centroid as the mean of its assigned points.
5. Repeat steps 3–4 until centers move very little or max_iter is reached.

K-Means attempts to minimize the total squared distance between each point and its assigned centroid.

This is called:

Within-Cluster Sum of Squares

Often represented as:

$$WCSS = \sum_{i=1}^{K} \sum_{x \in C_i} ||x - \mu_i||^2$$

where:

- K = number of clusters
- C_i = cluster i
- x = data point
- μ_i = centroid of cluster i

When K-means is a poor fit

- Elongated, crescent-shaped or otherwise non-convex groups.
- Clusters with very different densities or strongly unequal sizes.
- Data dominated by outliers, because means and squared distances are sensitive to extremes.
- Categorical data treated as arbitrary numbers.
- Mixed numeric/categorical data when ordinary Euclidean distance has no meaningful interpretation.
- Very high-dimensional data where distances become less discriminative; reduce dimensions or use a domain-appropriate representation.

Note: Scaling is part of the metric when dealing with distance like in KNN

Choosing K

WCSS (Within- Cluster Sum of Squares) always decreases as K increases.

If we simply select minimum WCSS (Within- Cluster Sum of Squares), we'd choose:

K = number of data points

which is useless.

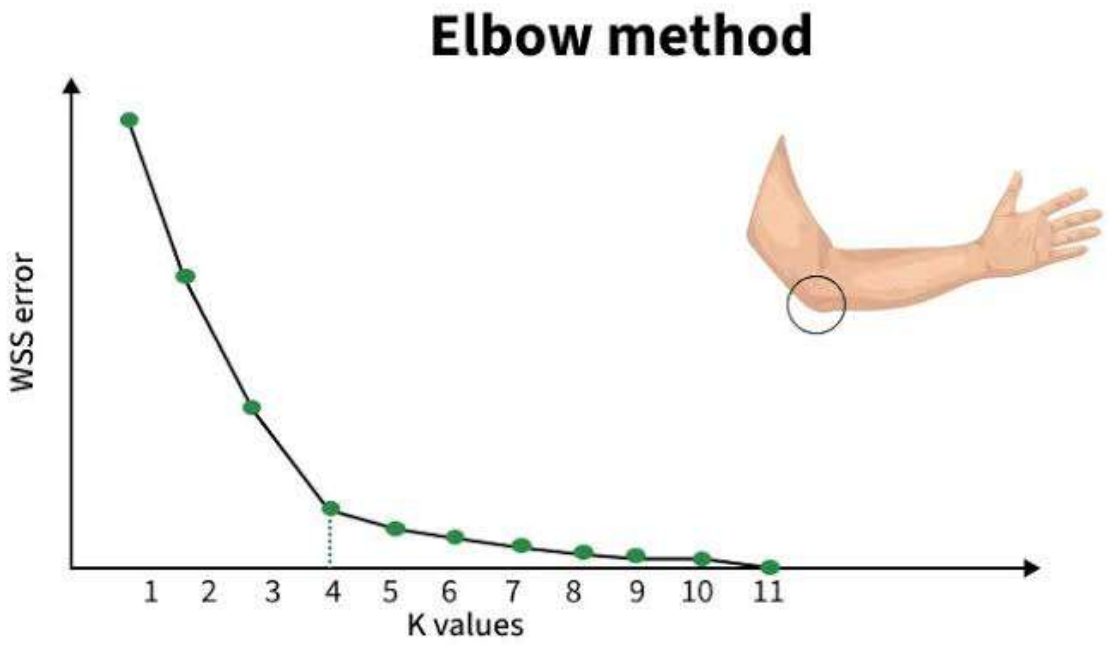
Therefore, we need a method to find a **reasonable K**.

Elbow method

The **Elbow Method** is a heuristic technique used to determine the optimal number of clusters (k) in partitioning algorithms like K-Means.

The idea is:

1. Run K-Means for different values of K.
2. Calculate WCSS for every K.
3. Plot: K vs WCSS



We look for the point where the curve starts becoming significantly flatter.

This is called the: **Elbow point**

Python — Elbow Method

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)


from sklearn.cluster import KMeans

import matplotlib.pyplot as plt

wcss = []
```

```
for k in range(1, 11):
    model = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    model.fit(X_scaled)
    wcss.append(model.inertia_)

plt.plot(range(1, 11), wcss, marker='o')
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.show()
```

Silhouette score

Sometimes using Elbow Method, we can't find the optimal value of k that's why we need another metric for this

Silhouette score measures how well a point fits within its cluster compared with other clusters.

Its value ranges approximately from: -1 to 1

Silhouette Score	**Interpretation**
Close to +1	**Very good clustering**
Around 0	**Overlapping clusters**
Negative	**Point may belong to another cluster**

For every point, calculate two quantities.

A - Average distance from the point to other points in its own cluster.

B - Average distance from the point to points in the nearest other cluster.

Silhouette coefficient:

$$s = \frac{b - a}{\max(a, b)}$$

Practical lab: customer segmentation

Understanding Data

RFM = Recency + Frequency + Monetary

These are three customer-behaviour features.

1.) **Recency** → How recently did the customer purchase?

Lower Recency is better.

Because Recency represents the number of days since the last purchase.

2.) Frequency → How often does the customer purchase?
3.) Monetary → How much money has the customer spent?

For example:

(260, 12, 26, 4200)

means approximately:

260 customers

average Recency ≈ 12

average Frequency ≈ 26

average Monetary ≈ 4200

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA

# Synthetic customer-level RFM data
rng = np.random.default_rng(42)
parts = [(260, 12, 26, 4200), (380, 50, 13, 1800),
         (360, 130, 4, 650), (200, 250, 15, 2400)]
```

```
rows = []
for n, recency_mu, frequency_mu, monetary_mu in parts:
    recency = np.clip(rng.normal(recency_mu, recency_mu*.20, n), 1, None)
    frequency = np.clip(rng.lognormal(np.log(frequency_mu), .28, n), 1, None)
    monetary = np.clip(frequency * rng.lognormal(
        np.log(monetary_mu/frequency_mu), .25, n), 20, None)
    rows.extend(zip(recency, frequency, monetary))
rfm = pd.DataFrame(rows, columns=["Recency", "Frequency", "Monetary"])

# Transform skewed features, then scale them
X = np.log1p(rfm[["Recency", "Frequency", "Monetary"]])
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Select K using both diagnostics
ks = range(2, 9)
inertia, silhouette = [], []
models = {}
for k in ks:
    model = KMeans(n_clusters=k, init="k-means++", n_init=20, random_state=42)
    labels = model.fit_predict(X_scaled)
    models[k] = model
    inertia.append(model.inertia_)
    silhouette.append(silhouette_score(X_scaled, labels))

fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].plot(list(ks), inertia, marker="o"); ax[0].set_title("Elbow: inertia")
ax[0].set_xlabel("K"); ax[0].set_ylabel("Inertia")
ax[1].plot(list(ks), silhouette, marker="o", color="darkorange")
ax[1].set_title("Silhouette score"); ax[1].set_xlabel("K"); ax[1].set_ylabel("Mean score")
plt.tight_layout(); plt.show()

# Choose after reviewing the plots and business constraints
best_k = list(ks)[int(np.argmax(silhouette))]
model = models[best_k]
rfm["Cluster"] = model.labels_

# Profile in original units; cluster IDs are arbitrary
profile = rfm.groupby("Cluster").agg(
    Customers=("Cluster", "size"),
    Recency=("Recency", "mean"),
    Frequency=("Frequency", "mean"),
    Monetary=("Monetary", "mean")
)
profile["Share_%"] = 100 * profile["Customers"] / len(rfm)
print(profile.sort_values("Monetary", ascending=False).round(2))

# Visualize with PCA only for visualization, not as the business label
coords = PCA(n_components=2, random_state=42).fit_transform(X_scaled)
plt.scatter(coords[:, 0], coords[:, 1], c=rfm["Cluster"], cmap="tab10", s=14, alpha=.65)
plt.xlabel("PC1"); plt.ylabel("PC2"); plt.title(f"Customer segments, K={best_k}")
plt.show()
```

Real transaction data: RFM aggregation template

```
# Required columns: CustomerID, InvoiceDate, InvoiceNo, Quantity, UnitPrice
transactions["InvoiceDate"] = pd.to_datetime(transactions["InvoiceDate"])
transactions["Sales"] = transactions["Quantity"] * transactions["UnitPrice"]
valid = transactions[
    transactions["CustomerID"].notna() &
    (transactions["Quantity"] > 0) &
    (transactions["UnitPrice"] > 0)
].copy()
snapshot = valid["InvoiceDate"].max() + pd.Timedelta(days=1)
rfm = valid.groupby("CustomerID").agg(
    Recency=("InvoiceDate", lambda s: (snapshot - s.max()).days),
    Frequency=("InvoiceNo", "nunique"),
    Monetary=("Sales", "sum")
).reset_index()
```

Returns and cancellations: The example excludes negative quantity/price rows. In a real business, decide whether returns should reduce Monetary, create a separate return-rate feature, or be analyzed separately. Do not hide this decision.

Profiling and naming segments

Never name a cluster from its numeric ID. Compare each cluster with the overall median or mean, then use behavior-based names that communicate an observable pattern. For example, “recent, frequent, high-value” is more defensible than “premium people” until the business has validated that interpretation.

```
overall = rfm[["Recency", "Frequency", "Monetary"]].median()
profile = rfm.groupby("Cluster")[["Recency", "Frequency", "Monetary"]].median()
index_vs_overall = profile / overall
print(index_vs_overall.round(2))
```

A possible action map for the synthetic pattern is:

Observed profile	Working name	Possible next action	Measurement
Low recency, high frequency, high monetary	Champions / active high value	Loyalty recognition, early access, cross-sell without over-discounting.	Repeat rate, margin, cross-category adoption.
Medium recency, medium frequency and spend	Loyal regulars	Personalized recommendations and points/membership nudges.	Incremental orders, retention, average order value.
High recency, low frequency and spend	Occasional / dormant	Low-cost reminder, onboarding or category education; test incentive carefully.	Reactivation rate and cost per reactivated customer.
High recency but high historic spend/frequency	At-risk high value	Service recovery, personalized outreach, stronger retention treatment.	Reactivation, churn proxy, revenue or margin saved.

Causal caution: A cluster describes behavior; it does not prove that a particular campaign will work. Use randomized holdouts or controlled experiments to estimate incremental impact.

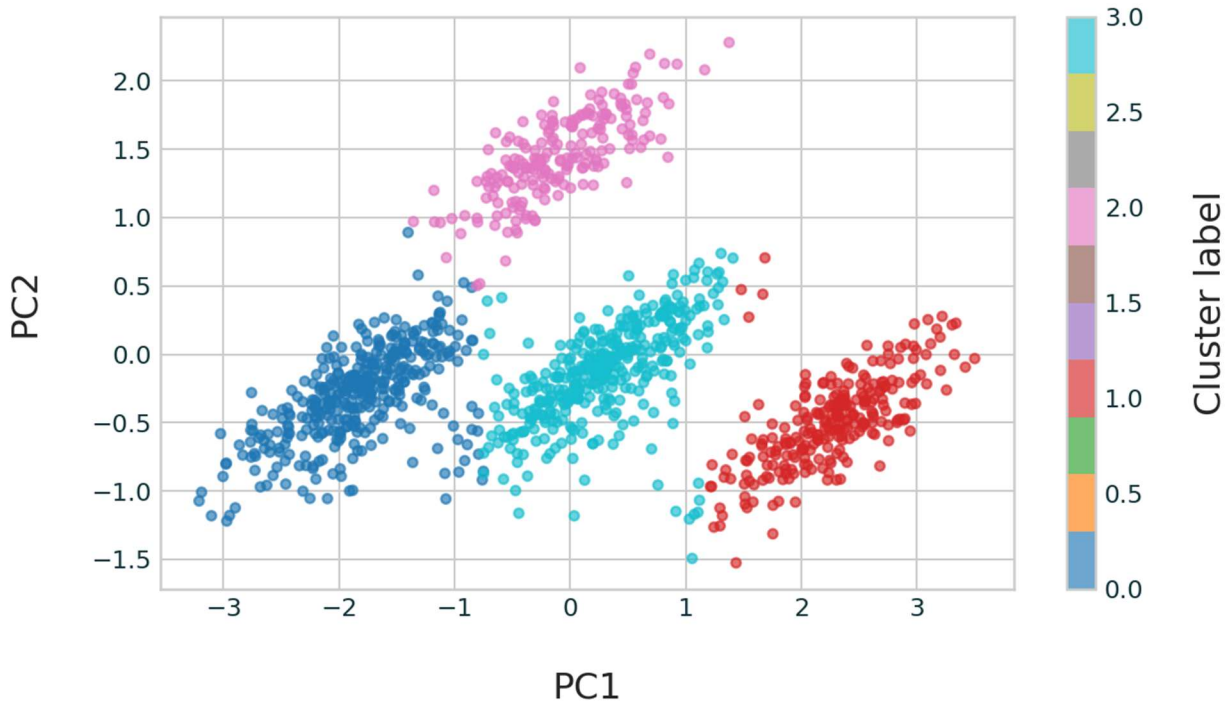


Figure 2. PCA projection for visual inspection only; the model was fitted on the scaled RFM features.